CommercialDetector — Detection System Overview

System Architecture

```
HDMI Source (TV)
       |
       v
USB Capture Device ──> Raspberry Pi 5 (or equivalent ARM SBC)
                                |
                 ┌──────────────┴──────────────┐
                 |                             |
         FFmpeg #1 (main)         FFmpeg #2 (audio-only)
         Video + Audio filters    16kHz mono PCM extraction
                 |                             |
                 v                             v
┌────────────────────────┐     ┌──────────────────────────┐
|     Signal Source      |     |   Transcript Analyzer    |
|  (stderr line parser)  |     | (faster-whisper + scorer)|
|                        |     |                          |
|  silencedetect -> sig  |     |  10s audio chunks        |
|  blackdetect  -> sig   |     |  → Whisper transcribe    |
|  scdet        -> sig   |     |  → KeywordScorer         |
|  ebur128      -> sig   |     |  → signal queue          |
└────────────────────────┘     └──────────────────────────┘
            |                               |
            |                               |
            v                               v
      ┌──────────────────────────────────────────┐
      |            Detection Engine              |
      |     (continuous multi-signal scoring)    |
      |                                          |
      |  score += signal_weight                  |
      |  score -= decay_rate × elapsed_time      |
      |  score clamped to [min_score, max_score] |
      |                                          |
      |  score >= 8.0  ──> COMMERCIAL            |
      |  score <= -3.0 ──> PROGRAM               |
      └──────────────────────────────────────────┘
                          |
                          |
                          v
                  ┌────────────────┐
                  | MQTT Publisher|
                  |  (paho-mqtt) |
                  └────────────────┘
                          |
                   MQTT Topics:
               commercial-detector/state          (retained)
               commercial-detector/state/changed   (event)
               commercial-detector/health          (heartbeat)
```

Signal Pipeline

The system extracts six signal types from two parallel FFmpeg processes, plus two transcript-derived signals from Whisper analysis.

FFmpeg #1 — Main Signal Source

A single FFmpeg subprocess applies detection filters to the input stream and writes filter events to stderr in real time.

Filter chain:

Filter	Stream	Output
`silencedetect`	Audio	`SILENCE_START`, `SILENCE_END` with duration
`ebur128`	Audio	Momentary + short-term LUFS at ~10 Hz
`blackdetect`	Video	`BLACK_START` with duration
`scdet`	Video	`SCENE_CHANGE` with score

FFmpeg command (live capture):

```
ffmpeg -hide_banner \
  -f v4l2 -input_format mjpeg -i /dev/video0 \
  -f alsa -i hw:1 \
  -vf "blackdetect=d=0.05:pix_th=0.10,scdet=threshold=10.0" \
  -af "silencedetect=noise=-42.0dB:d=0.5,ebur128=peak=true" \
  -f null -
```

Stderr lines are parsed by regex matchers in `signal_source.py`. Each recognized line becomes a `DetectionSignal(timestamp, signal_type, value)`.

FFmpeg #2 — Transcript Audio Extraction

A second FFmpeg subprocess extracts audio as 16 kHz mono signed 16-bit PCM, piped to stdout:

```
ffmpeg -hide_banner -loglevel error \
  -i <input> \
  -vn -ar 16000 -ac 1 -f s16le pipe:1
```

The `TranscriptAnalyzer` reads 10-second chunks from this pipe, feeds them to `faster-whisper` (tiny.en model, int8 quantization, beam_size=1, VAD filter enabled), and scores the resulting text with `KeywordScorer`.

Signal Types

Signal	Source	Weight	Meaning
`SILENCE_END`	silencedetect	**+5.0**	Silence gap ended — boundary between ads or between ads and program
`BLACK_START`	blackdetect	**+4.0**	Black frame detected — ad segment boundary marker
`LOUDNESS_SHIFT` (up)	ebur128	**+4.0**	Volume jump above baseline — ad starting (CALM Act: ads are louder)
`LOUDNESS_SHIFT` (down)	ebur128	**-3.0**	Volume drop below baseline — program resuming at normal level
`SCENE_CHANGE`	scdet	**+0.5**	Individual scene cut (weak alone; strong when rate is high)
`TRANSCRIPT_COMMERCIAL`	Whisper + scorer	**+3.0**	Whisper heard commercial language ("call now", pricing, etc.)
`TRANSCRIPT_PROGRAM`	Whisper + scorer	**-4.0**	Whisper heard natural dialog without commercial indicators
`SILENCE_START`	silencedetect	**0**	Informational only — no score contribution

Detection Engine — Scoring Model

The engine maintains a single `commercial_score` that moves up with commercial-like evidence and down with program-like evidence plus continuous time decay.

Score Lifecycle

```
+15 — max_score ———————————————————— ceiling clamp
      |
      |   ▲ silence+black coincidence (13.5)
      |   |
+8  —|—— commercial_threshold ———————— COMMERCIAL triggered
      |
 0  —|————————————————————————————  neutral
```

```
       |
  -3  —|—— program_threshold ———————— PROGRAM triggered
       |
 -10 — min_score ——————————————————— floor clamp
```

Score Accumulation

Each signal adds its weight to `commercial_score`:

```
score += signal_weight
score = clamp(score, min_score, max_score)
```

Score Decay

Between signals, the score decays toward zero:

```
decay = score_decay_rate × elapsed_seconds    (0.6 pts/sec)

if score > 0:  score = max(0, score - decay)
if score < 0:  score = min(0, score + decay)
```

A score at `max_score` (15.0) decays to zero in 25 seconds of silence — tuned to match the typical 25-second post-break gap where no FFmpeg signals arrive.

Coincidence Detection

When `SILENCE_END` and `BLACK_START` occur within 2.0 seconds of each other, they are treated as a **coincident boundary signal**. This is the strongest commercial break indicator.

- Both signals receive a **1.5x multiplier** (when not already in COMMERCIAL state)
- Combined score: (5.0 + 4.0) x 1.5 = **13.5 points** from a single coincident pair
- This exceeds the 8.0 commercial threshold in a single event, enabling 2-3 second entry

State-aware behavior: When already in COMMERCIAL state, coincidence multiplier is disabled (set to 1.0). Ad-to-ad boundaries within a break are correctly classified but don't over-inflate the score, allowing faster exit when the break ends.

Scene Change Rate Boost

Individual scene changes are weak (+0.5). But commercials typically have rapid cuts (15-30+ per minute vs 3-10 for program content).

The engine tracks scene change timestamps in a 30-second sliding window: - If the rate exceeds **12 changes/minute**, an additional **+2.0 boost** is applied - This compounds with individual scene change scores during commercial segments

Transition Logic

```
PROGRAM or UNKNOWN → COMMERCIAL:   when score >= 8.0
COMMERCIAL → PROGRAM:              when score <= -3.0
```

Both transitions are gated by minimum dwell time (debounce): - Must stay in COMMERCIAL for at least **15 seconds** before exiting - Must stay in PROGRAM for at least **60 seconds** before re-entering

Score Clamping

The score is hard-clamped to `[min_score, max_score]` = `[-10, 15]` after every signal.

This prevents runaway accumulation during long commercial breaks where many reinforcing signals arrive. Without clamping, the score could reach 40+ during a 3-minute break, requiring 70+ seconds of pure decay to return to the program threshold. With clamping at 15, the maximum decay time is bounded.

Safety math: During a break, the longest gap between reinforcing signals (silence/black/scene events) is ~10 seconds. In that gap: `15 - (10 × 0.6) - 4.0 = 5.0` (worst case: one false transcript_program of −4.0). Since 5.0 > −3.0 (program threshold), the system never falsely exits during a break.

Initial Hold-Off

The first 60 seconds of the stream are ignored for scoring (signals are parsed but don't accumulate score). This avoids false triggers from intro sequences, channel idents, or initial stream noise.

Transcript Analysis

Whisper Configuration

Parameter	Value	Rationale
Model	`tiny.en`	Fastest on ARM; ~550 MB total with Whisper on RPi 5
Compute type	`int8`	Quantized for ARM efficiency
Beam size	`1`	Greedy decoding — fastest inference
VAD filter	`true`	Skip chunks with no speech
Chunk duration	`10s`	Reliable program dialog detection (25+ words at normal speech rate)
Sample rate	`16 kHz`	Whisper requirement

KeywordScorer

The scorer classifies each transcript chunk as commercial, program, or inconclusive using two complementary strategies:

Commercial detection — 45+ regex patterns with weights (1.0-3.0):

Category	Example Patterns	Weight
Direct response	“call now”, “act now”, “order now”	2.5-3.0
Pricing	“$19.99”, “50% off”, “free shipping”	1.5-2.5
Contact/web	“1-800-...”, “visit .com”, “www.”	1.5-3.0
Medical/pharma	“side effects”, “ask your doctor”	3.0
Sponsorship	“brought to you by”, “proud sponsor”	2.0-2.5
Security/home	“24/7 monitoring”, “protect your home”	1.5-2.5
Healthcare	“highly skilled doctors”, “dental care”	1.5
Insurance	“get a free quote”, “get approved”	2.0
Urgency	“while supplies last”, “limited time”	2.0

A chunk is classified `TRANSCRIPT_COMMERCIAL` when the cumulative pattern score >= 2.0.

Program detection — word count heuristic: - If a chunk has **25+ words** and a commercial pattern score < **1.0**, it’s classified as natural dialog (`TRANSCRIPT_PROGRAM`) - Program score scales with word count: `min(word_count / 25, 3.0) × 2.0`

Transcript Timestamp Synchronization

In file mode, the audio-only FFmpeg #2 runs faster than the main FFmpeg #1 (which processes video filters). Transcript signals arrive with timestamps in the “future” relative to the main signal stream.

The main loop holds transcript signals in a pending buffer and releases them only when the main FFmpeg’s current PTS catches up:

```
# Release pending transcript signals whose timestamp <= current PTS
for ts in _pending_transcript:
    if ts.timestamp <= current_pts:
        engine.process(ts)
```

This prevents out-of-order timestamps from corrupting the engine’s decay calculations.

Loudness Detection (EBU R128)

The ebur128 filter measures loudness per EBU R128 standard, outputting ~10 measurements per second. Only a small fraction become detection signals.

Baseline Tracking

An exponential moving average (EMA) tracks the long-term loudness baseline:

```
baseline += alpha × (short_term_LUFS - baseline)
```

Where `alpha = 0.02` (slow adaptation — ~50 second effective window).

Shift Detection

When **short-term LUFS** (3-second integration window) deviates from baseline by more than **5.0 LUFS**, a `LOUDNESS_SHIFT` signal is emitted.

- Positive delta → volume jump → `loudness_up_weight` (+4.0) → commercial evidence
- Negative delta → volume drop → `loudness_down_weight` (-3.0) → program evidence
- After emission, baseline resets to current short-term LUFS to avoid repeated triggers
- **5-second debounce** prevents signal flooding (minimum 5s between emissions)

Why Short-Term LUFS (Not Momentary)

The EBU R128 standard defines two real-time measurements: - **Momentary** (M): 400 ms integration — very responsive but captures transient fluctuations within content - **Short-term** (S): 3 s integration — stable, captures actual boundary-level changes

Testing showed momentary LUFS generated ~250 signals per 28-minute episode (many noise triggers within commercial breaks), while short-term LUFS generates ~10 — only at genuine content boundaries. The momentary signals kept the score elevated during breaks, delaying exit detection.

MQTT Output

Topics

Topic	Retain	Content	Example
`commercial-detector/state`	Yes	Current state string	`"commercial"`
`commercial-detector/state/changed`	No	Transition JSON	`{"timestamp": 354.9, "from": "program", "to": "commercial", ...}`
`commercial-detector/health`	No	Heartbeat JSON	`{"timestamp": 1709510000.0, "state": "commercial"}`

Transition Event Payload

```
{
  "timestamp": 354.9,
  "from": "unknown",
  "to": "commercial",
  "confidence": 0.713,
  "signals": {
    "commercial_score": 11.41,
    "trigger_signal": "black_start",
    "score_delta": 4.0
  }
}
```

Heartbeat

Published every 30 seconds. Can be used by downstream systems to verify the detector is alive.

Performance — Measured Results

Tested against a 28-minute TV episode (S2E27 "Pulse Of The Pack") with three known commercial breaks. All features enabled (FFmpeg filters + Whisper transcript analysis).

Ground Truth

Break	Actual Start	Actual End	Duration
Break 1	~347s (5:47)	~510s (8:30)	~163s (2:43)
Break 2	~905s (15:05)	~1055s (17:35)	~150s (2:30)
Break 3	~1415s (23:35)	~1583s (26:23)	~168s (2:48)

Detection Results

Break	Entry Time	Entry Latency	Exit Time	Exit Latency	Entry Trigger
Break 1	354.9s	**+7.9s**	520.0s	**+10.0s**	`black_start` (coincidence)
Break 2	910.8s	**+5.8s**	1060.0s	**+5.0s**	`scene_change` (rate boost)
Break 3	1423.9s	**+8.9s**	1590.0s	**+7.0s**	`black_start` (coincidence)

Summary: - Entry latency: **5.8-8.9 seconds** (target: <10s) - Exit latency: **5.0-10.0 seconds** (target: <15s) - False exits during breaks: **0** - False positive at video end: **1** (final content misclassified — edge case)

Signal Volume (28-minute episode)

Signal Type	Count	Notes
`transcript_program`	230	Primary exit driver
`transcript_commercial`	42	Reinforces scoring during breaks
`loudness_shift`	19	Boundary-level changes only
`scene_change`	~800+	Individual weak; rate boost is meaningful
`silence_end`	~20-30	Primary entry driver (with black coincidence)
`black_start`	~15-20	Primary entry driver (with silence coincidence)

Entry Mechanism — How It Works

1. Commercial break begins with a **silence gap** + **black frame** at the program/ad boundary
2. These typically occur within 2 seconds of each other ⟶ **coincidence detected**
3. Combined score: `(5.0 + 4.0) × 1.5 = 13.5` ⟶ exceeds 8.0 threshold immediately
4. Transition fires in **2-3 seconds** from the first boundary marker

For breaks without clear coincident signals (Break 2), accumulation from **scene change rate boost** (rapid cuts in ads) + individual signals reaches the threshold in **5-10 seconds**.

Exit Mechanism — How It Works

1. Commercial break ends — last ad's boundary markers push score toward `max_score` (15.0)
2. After the break, no silence/black/scene signals arrive for ~25 seconds
3. Score decays: `15.0 - (25 × 0.6) = 0.0` — score reaches zero from decay alone
4. Meanwhile, **`TRANSCRIPT_PROGRAM` signals** (-4.0 each) arrive every ~10 seconds as Whisper recognizes program dialog
5. First transcript_program after score reaches ~1.0 pushes it below the −3.0 program threshold
6. Transition fires — typically **5-10 seconds** after the break ends

Without transcript analysis, exit relies solely on decay, taking ~30 seconds.

Confidence Calculation

Confidence reflects how far the score exceeds the threshold:

```
For COMMERCIAL:  0.5 + (score - 8.0) / 16.0    (clamped to [0.1, 1.0])
For PROGRAM:     0.5 + (-3.0 - score) / 6.0     (clamped to [0.1, 1.0])
```

Higher confidence indicates the score exceeded the threshold by a wider margin.

Configuration Reference

All parameters are tunable via `config.yaml`. Current production defaults:

Signal Source (FFmpeg Filters)

Parameter	Default	Description
`silence_noise_db`	−42.0 dB	Noise floor — filters out dialog pauses
`silence_duration`	0.5 s	Minimum silence gap to detect
`black_min_duration`	0.05 s	Minimum black frame duration
`black_pixel_threshold`	0.10	Pixel brightness threshold (0-1)
`scene_change_threshold`	10.0	scdet score threshold

Scoring Engine

Parameter	Default	Description
`initial_hold_off`	60.0 s	Ignore signals before this time
`min_commercial_duration`	15.0 s	Debounce: minimum time in COMMERCIAL
`min_program_duration`	60.0 s	Debounce: minimum time in PROGRAM
`silence_weight`	+5.0	Score for SILENCE_END
`black_frame_weight`	+4.0	Score for BLACK_START
`loudness_up_weight`	+4.0	Score for loudness increase
`loudness_down_weight`	−3.0	Score for loudness decrease
`scene_change_weight`	+0.5	Score for individual scene change
`transcript_commercial_weight`	+3.0	Score for commercial transcript
`transcript_program_weight`	−4.0	Score for program transcript
`coincidence_window`	2.0 s	Max gap for silence+black coincidence
`coincidence_multiplier`	1.5x	Boost for coincident signals
`score_decay_rate`	0.6 pts/s	Decay speed toward zero
`commercial_threshold`	+8.0	Score to enter COMMERCIAL
`program_threshold`	−3.0	Score to enter PROGRAM
`max_score`	+15.0	Score ceiling clamp
`min_score`	−10.0	Score floor clamp

Loudness

Parameter	Default	Description
`loudness_shift_db`	5.0 LUFS	Deviation from baseline to emit signal
`loudness_ema_alpha`	0.02	EMA smoothing factor (~50s effective window)

Scene Change Rate

Parameter	Default	Description
`scene_rate_window`	30.0 s	Sliding window for rate calculation
`scene_rate_threshold`	12.0/min	Rate threshold for boost
`scene_rate_boost`	+2.0	Additional score when rate exceeds threshold

Transcript Analysis (Optional)

Parameter	Default	Description
`enabled`	false	Requires `pip install faster-whisper`
`whisper_model`	"tiny.en"	Model size (tiny.en recommended for SBC)
`chunk_duration`	10.0 s	Audio chunk size for Whisper
`commercial_threshold`	2.0	Min pattern score for TRANSCRIPT_COMMERCIAL
`program_threshold`	2.0	Min word-count score for TRANSCRIPT_PROGRAM
`program_min_words`	25	Words without commercial indicators → program

Hardware Requirements

Minimum: Raspberry Pi 5 (4 GB)

Component	Memory	CPU	Notes
FFmpeg #1 (video+audio filters)	~100 MB	Moderate	H.264 decode + 4 filters
FFmpeg #2 (audio extraction)	~30 MB	Low	Audio-only, no decode
faster-whisper (tiny.en, int8)	~400 MB	Moderate	RTF 0.23-0.41 on RPi 5
Python + detection engine	~50 MB	Low	Event-driven, no buffering
Total	**~580 MB**		Fits in 4 GB with headroom

- Active cooling recommended (fan or heatsink) for sustained operation
- Without Whisper: ~180 MB total (comfortably runs on 2 GB)
- Whisper processes 10s chunks faster than real-time (RTF < 1.0 on RPi 5)

Tuning History

The scoring parameters went through iterative tuning. Key evolution:

Version	max_score	decay_rate	Exit Latency (avg)	Issue
v1	100	0.5	45-90s	No clamping — score runaway to 40+
v2	20	0.5	20-30s	Clamping added but max too high
v3	20	0.5	20-30s	Same as v2 (lower max didn't help)
v4	15	0.6	**5-10s**	Mathematically derived — current production

v4 tuning math: - Safety: `15 - (10 × 0.6) - 4 = 5 > -3.0` ⟶ survives 10s in-break signal gap + false transcript - Speed: `15 - (25 × 0.6) = 0` ⟶ score reaches zero in exactly 25s (post-break FFmpeg gap) - First `transcript_program` (-4.0) after zero ⟶ crosses -3.0 threshold ⟶ exit

Dead Ends in Tuning

- **Momentary LUFS** (400 ms): Generated 25x more loudness events than short-term LUFS. Many were transient fluctuations within commercial breaks, keeping the score elevated and delaying exit. Reverted to short-term LUFS (3s window).
- **5-second Whisper chunks**: Halved the available speech context, causing 93% fewer `transcript_program` signals (9 vs 121). Even with a proportionally scaled word threshold (12 vs 25), only 18 signals were generated. Reverted to 10-second chunks.
- **Aggressive decay** (1.0 pts/s): Caused false exits during breaks when conversational commercials were misclassified as program content by Whisper. The score dropped too quickly to absorb false negatives.